

Lab 6: Design and FPGA Implementation of the ALU for a Single-Cycle RISC-V Processor

Objective

To design and implement the ALU for a single-cycle RISC-V processor (RV32I base extension) using Verilog HDL, verify its functionality through a simulation testbench, synthesize the design on an FPGA, and demonstrate its operation using an FSM-based verification approach. The ALU should support the following core RISC-V operations:

ADD, SUB, SLL, SRL, AND, OR, XOR, BEQ

Task 1: ALU Block Diagram Design

Objective

To create a block diagram of the ALU showing the inputs, outputs, and internal operations.

Procedure

- (a) Design the 1-bit ALU block diagram showing:
 - Arithmetic operations: ADD, SUB
 - Logic operations: AND, OR, XOR
 - Shift operations: SLL, SRL
 - Input multiplexers (if needed for operand selection)
- (b) Design the 32-bit ALU block diagram showing:
 - Arithmetic operations: ADD, SUB
 - Logic operations: AND, OR, XOR
 - Shift operations: SLL, SRL
- (c) Draw clear diagrams showing data flow from inputs through the operation blocks to outputs.

Deliverables

- Block diagram of the ALU.

Task 2: Verilog Description and Testbench Development

Objective

Implement the ALU in Verilog based on the block diagram and verify its functionality using simulation.

Procedure

- (a) Implement the ALU module in Verilog:
 - Inputs: A, B, ALUControl[3:0]
 - Outputs: ALUResult, Zero
- (b) Follow the block diagram to implement arithmetic, logic, and shift operations.
- (c) Develop a testbench:
 - Apply test vectors for all operations (ADD, SUB, AND, OR, XOR, SLL, SRL, etc.).
 - Verify outputs match the expected results.
 - Generate waveforms for visual verification.
- (d) Run simulation to ensure correctness of all ALU operations.

Deliverables

- Verilog source file for ALU (ALU.v)
- Testbench file (ALU_tb.v)
- Simulation waveforms showing correct operation

Task 3: FPGA Synthesis and RTL Analysis

Objective

Synthesize the ALU design on an FPGA, analyze the RTL structure, and verify functionality using switches and LEDs.

Procedure

- (a) Integrate ALU module into a top-level design.
- (b) Implement a simple FSM to cycle through ALU operations.
- (c) Map ALUControl input to FPGA switches.
- (d) Map ALU outputs (ALUResult, Zero) to LEDs.
- (e) Use two fixed 32-bit numbers (e.g., 0x10101010, 0x01010101) as operands A and B.
- (f) Use the FSM to read switch inputs and display ALU results on LEDs.
- (g) Observe LED patterns to confirm correct ALU operations.
- (h) Synthesize the design and verify the RTL schematic.
- (i) Program the FPGA, toggle switches, and verify LEDs reflect expected outputs.



Deliverables

- RTL schematic screenshots
- Demonstration of FPGA operation to RA for approval

Note: The switch interface, button debouncing module, and LED interface must be re used from Lab 5.



Assessment Rubric

Lab 6: Design and FPGA Implementation of the ALU for a Single-Cycle RISC-V Processor

Name:	Student ID:	Section:
--------------	--------------------	-----------------

Points Distribution

	Task No.	LR 2 (Code)	LR 5 (Results)	LR 11 (Design)
In-Lab	Task 1	-	-	/20
	Task 2	/20	/15	-
	Task 3	/10	/15	-
Total Points		/30	/30	/20
CLO Mapped		CLO 1		

Affective Domain Rubric		Points	CLO Mapped
AR7	Report Submission & Git Upload	/10 & /10	CLO 1

CLO	Total Points	Points Obtained
1	100	
Total	100	

For description of different levels of the mapped rubrics, please refer to the Lab Evaluation Assessment Rubrics and Affective Domain Assessment Rubrics provided [here](#).